K GraphFuse: Similarity-Aware Sparse Polynomial Evaluation with Selective GPU Compensation

Utkarsh Sinha

Technical research manuscript / preprint draft, 10 October 2026; not peer-reviewed.

Download the PDF · Standalone LaTeX source · Metric definitions

Abstract

We study a source-separated prototype for evaluating finite matrix-valued polynomials of a sparse graph operator and their directional derivatives. The method combines exact ordered coefficient fusion, a fixed dyadic diagonal similarity, fused sparse Horner evaluation, and profile-specific floating-point compensation. The mathematical contribution is an explicit transfer bound: if the retained computation approximates the polynomial in balanced coordinates with induced Frobenius error at most δ_B, then the corresponding output in the original coordinates has error at most $\kappa_2(D)\delta_B\|X\|_F$. We prove this by similarity and submultiplicativity; the numerical-range tail planner uses a complete Crouzeix inequality as a stated external theorem premise, not as a theorem independently proved here. We also give the coupled Horner recurrence for a diagonal affine derivative and an exact-rational coefficient-encoding bound.

A constructed alternating sparse family was evaluated on one Tesla T4 across six resident workloads with 4 or 16 channels. Against independently tuned FP64 CuPy or PyTorch controls, all six selected profiles passed the recorded sampled-error and latency gates. Pooled-median latency reductions were 19.98%–73.26%; the corresponding reciprocal-latency rate increases were 24.98%–273.95%. These are two descriptions of the same measured latency ratios, not independent throughput measurements. Every complete output was compared with an FP64 CPU computation; 256/384-bit checks covered only the first 16 rows of the large fixtures. Results are near the FP64 rounding floor and are limited by one constructed family, one GPU architecture, post-exploration profile selection, and within-run uncertainty. This study does not compare these polynomial cases with native cuGraph, establish broad graph-algorithm performance, or change the installed K GraphFuse core.

Keywords: sparse graph filters; matrix polynomials; similarity scaling; numerical range; Horner evaluation; directional sensitivity; compensated arithmetic; CUDA

1. Introduction

Many graph workloads apply a sequence of linear sparse propagations and small channel transforms to a feature matrix. When the graph operator is fixed and the stages are linear, the ordered stages can be represented by a single matrix-valued polynomial. Evaluation then becomes a sequence of sparse matrix–feature products plus dense transformations on a small channel dimension. This is a useful setting for fusing memory-bound sparse work while retaining the full finite polynomial.

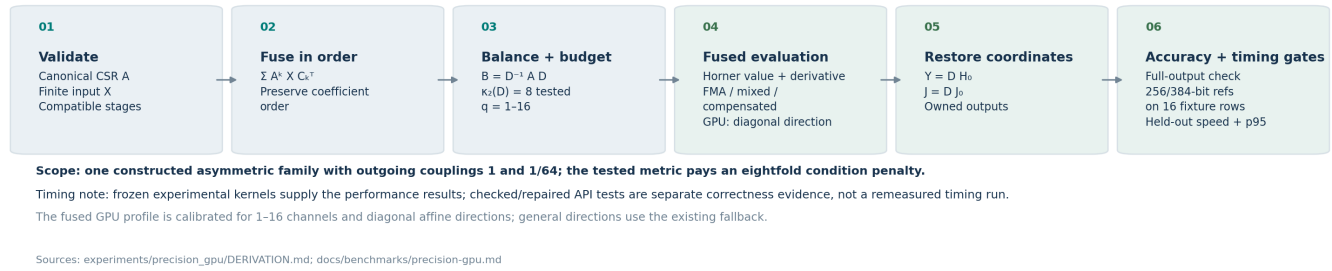
This manuscript describes K GraphFuse’s experimental precision-GPU prototype [8]. It asks a narrow question: for one deliberately asymmetric sparse family, can a simple diagonal coordinate change reduce the polynomial degree needed by an existing error planner, while fused evaluation and selected compensation preserve measured accuracy and improve resident latency against FP64 framework controls?

The answer is positive for the six reported T4 cases, under a post-exploration selection procedure and the exact timing and error contracts below. The result is not a general claim about graph workloads. The

balancing rule is a one-step heuristic, not an optimizer. The theorem we prove transfers any balanced-coordinate approximation bound back to the original coordinates. The numerical-range tail bound relies on a cited complete Crouzeix inequality supplied as a premise. This work neither proves that inequality nor verifies every premise in its separate proof.

The implementation is maintained under *experiments/precision_gpu/*, separately from the installed *k_graphfuse* 0.1.0 numerical core. The tested core is unchanged. Timed results belong to the frozen *source/product_v1* and *source/wide_v1* sources; later checked-API repairs passed tests but were not given a new full performance run.

Figure 1. *Pipeline overview.*



1.1 Contributions

This manuscript makes four bounded contributions.

1. It states and proves the original-coordinate error-transfer theorem for a fixed positive diagonal similarity.
2. It writes the exact coupled Horner recurrence for a value and its derivative with respect to a fixed affine graph parameter, and gives an exact-rational coefficient-encoding certificate.
3. It describes the fused sparse GPU evaluation and its arithmetic and workspace costs without claiming an asymptotic graph-algorithm improvement.
4. It reports six preselected T4 comparison cases with absolute medians, p95 values, derived latency and reciprocal-latency changes, sampled accuracy gates, source versions, and limitations.

The prototype does not introduce a new general sparse graph algorithm, an optimal balancing method, an adjoint engine, or a new proof of a spectral-set inequality.

2. Problem and notation

Let $A \in \mathbb{R}^{n \times n}$ be the stored sparse operator and $X \in \mathbb{R}^{n \times q}$ a feature matrix. Let $C_k \in \mathbb{R}^{q \times q}$, $k = 0, \ldots, d$, be the exactly fused, ordered coefficient matrices. The target is

$$P_A(X) = \sum_{k=0}^{d} A^k X C_k^{\mathsf{T}}.$$

The coefficient matrices need not commute. The polynomial representation therefore preserves the order induced by the original linear stages. Under row-major vectorization, the corresponding feature-space operator is $\sum_k A^k \otimes C_k$, whose Euclidean norm induces the Frobenius norm on feature matrices. We measure value error in Frobenius norm, normalized by $\|X\|_F$. This is not a per-vertex relative-error contract and is not aggregate PageRank L_1 error.

For a fixed sparse direction E, define $A(\theta) = A + \theta E$ on a specified interval $|\theta| \le h$, with C_k and X fixed. The directional sensitivity is

$$J_A(X;\theta) = \frac{d}{d\theta} P_{A(\theta)}(X).$$

It is not an adjoint gradient or a nonlinear graph-learning derivative.

3. Similarity transform and error transfer

Choose a fixed positive diagonal matrix D, and define $B = D^{-1}AD$. Then $A^k = DB^kD^{-1}$, so termwise

$$P_A(X) = D\,P_B(D^{-1}X).$$

Right multiplication by C_k^{T} does not alter the coordinate-change identity.

Theorem 1: transfer of a balanced-coordinate approximation bound

Suppose a linear evaluator $\widehat{P}_B$ satisfies, for every $Z \in \mathbb{R}^{n\times q}$,

$$\|P_B(Z) - \widehat{P}_B(Z)\|_F \le \delta_B \|Z\|_F.$$

Define the original-coordinate evaluator $\widehat{P}_A(X) = D\widehat{P}_B(D^{-1}X)$. Then

$$\begin{aligned}\|P_A(X) - \widehat{P}_A(X)\|_F \\ \le \kappa_2(D)\,\delta_B\,\|X\|_F,\end{aligned}$$

$$\kappa_2(D) = \|D\|_2\|D^{-1}\|_2.$$

Proof. The exact similarity identity gives

$$\begin{aligned}P_A(X) - \widehat{P}_A(X) \\ = D\big(P_B(D^{-1}X) - \widehat{P}_B(D^{-1}X)\big).\end{aligned}$$

Apply $\|DY\|_F \le \|D\|_2\|Y\|_F$, then the assumed bound with $Z = D^{-1}X$, and finally $\|D^{-1}X\|_F \le \|D^{-1}\|_2\|X\|_F$. Multiplying the factors yields $\|D\|_2\|D^{-1}\|_2\delta_B\|X\|_F$. \(\square\)

Thus an original-coordinate normalized error budget ε requires a balanced-coordinate budget no larger than $\varepsilon/\kappa_2(D)$. The result is independent of how D was selected. It applies to any operator-norm approximation bound satisfying the theorem's hypothesis, including a truncation or coefficient-encoding bound. Floating-point execution error is additional unless separately bounded.

3.1 Numerical-range tail bound as a conditional premise

For a matrix B, its numerical range is $W(B) = \{u^*Bu : \|u\|_2 = 1\}$. Let $Q_m(z) = \sum_{k=0}^{m} C_k z^k$ be the retained polynomial and let a_k upper-bound $\|C_k\|_2$. If $W(B)$ lies in an origin-centered disk of radius r, a complete spectral-set inequality with constant $K = 2$ gives the conditional tail bound

$$\begin{aligned}\Big\|\sum_{k>m} B^k \otimes C_k\Big\|_2 \\ \le 2\sum_{k>m} a_k r^k.\end{aligned}$$

A classical operator-norm enclosure $\rho \ge \|B\|_2$ also gives $\|\sum_{k>m} B^k \otimes C_k\|_2 \le \sum_{k>m} a_k \rho^k$. The planner takes the smaller valid bound. The use of $K = 2$ takes the complete Crouzeix inequality in OpenAI manuscript 325-01 as an

external theorem premise [3]. The proof of that premise was not independently rebuilt or checked for this manuscript. The similarity transfer in Theorem 1 is proved here; the heuristic choice of D does not change the theorem's validity.

3.2 Implemented dyadic metric

Let r_i and c_i be the absolute off-diagonal row and column sums of A. For indices where both are positive, the implementation sets

$$e_i = \left\lceil \frac{\log_2(r_i) - \log_2(c_i)}{4} \right\rceil,$$

$$D_{ii} = 2^{e_i - \max_j e_j}.$$

If either sum is zero, it uses $e_i = 0$. This is one parallel balancing step. The exponent rule is heuristic and is not an iterative optimizer. The condition penalty is computed as $2^{\max_i e_i - \min_i e_i}$. The transformed sparse entries are scaled by binary exponents and scaled back; nonfinite values, underflow, or failure of exact round-trip equality reject the transform.

In the tested alternating directed family, off-diagonal couplings are 1 and $1/64$. The chosen scale alternates between 1 and $1/8$, equalizing the two transformed couplings at $1/8$. The resulting $\kappa_2(D) = 8$ incurs an eightfold error-budget penalty. The narrower balanced numerical-range enclosure nevertheless lets the planner retain fewer polynomial terms for this construction.

4. Coupled value and derivative evaluation

Hold D fixed while $A(\theta) = A + \theta E$ varies. Write $Z = D^{-1}X$, $B(\theta) = D^{-1}A(\theta)D = B + \theta F$, and $F = D^{-1}ED$. Then

$$\frac{d}{d\theta} P_{A(\theta)}(X) = D \frac{d}{d\theta} P_{B(\theta)}(Z).$$

The value and derivative budgets are both divided by $\kappa_2(D)$.

For retained degree d, initialize $H_d = ZC_d^{\mathsf{T}}$ and $J_d = 0$. For $k = d-1, \ldots, 0$, using the previous states before overwriting,

$$J_k = B(\theta)J_{k+1} + FH_{k+1},$$
$$H_k = B(\theta)H_{k+1} + ZC_k^{\mathsf{T}}.$$

Since H_d is independent of θ, $J_d = dH_d/d\theta$. If $J_{k+1} = dH_{k+1}/d\theta$, the product rule gives

$$\frac{dH_k}{d\theta} = FH_{k+1} + B(\theta)\frac{dH_{k+1}}{d\theta} = FH_{k+1} + B(\theta)J_{k+1} = J_k.$$

Induction proves the recurrence at every degree, and the returned outputs are DH_0 and DJ_0. The recurrence preserves coefficient order and requires no commutativity assumption. For the tested diagonal direction with entries ± 1, $F = E$, so the directional product is elementwise. A general sparse F requires another sparse multiplication and uses the existing fallback path. Floating checks were made at $\theta = 0$ and both interval endpoints; those samples do not validate every floating output throughout the interval.

5. Coefficient encoding and arithmetic

For an exact rational coefficient matrix C_k, let H_k be its elementwise round-to-nearest binary64 encoding. An optional low part is $L_k = \mathrm{round}_{64}(C_k - H_k)$; high-only execution sets $L_k = 0$. Define the exact rational remainder $R_k = C_k - H_k - L_k$, and let a_k be an exact rational upper bound on $\|R_k\|_2$. If $\rho \geq \|B\|_2$ and $W(B)$ lies in a disk of radius r, the encoded-polynomial error obeys

$$\left\| \sum_k B^k \otimes R_k \right\|_2 \leq \min\left\{ \sum_k a_k \rho^k, \quad 2\sum_k a_k r^k \right\}.$$

The original-coordinate bound multiplies this by $\kappa_2(D)$. The derivative's encoding error is bounded separately by the existing directional-derivative polynomial bound, with the parameter interval included in its operator and numerical-range enclosures. These bounds account for exact polynomial truncation and coefficient representation. They do not account for CUDA execution roundoff, overflow, or underflow.

The narrow CUDA kernel assigns work to row/channel outputs and combines CSR edge accumulation, coefficient injection, and (when requested) the directional derivative. It uses alternating state buffers. FMA-based, compensated, mixed, and double-double profiles are available in the experimental source. Plain FMA uses one rounding for a product-plus-sum. Compensation tracks product and addition residuals; the compensated profile still rounds completed per-thread sums at stages. No profile eliminates all rounding or removes overflow and underflow risk.

For wide features, a BLAS projection forms ZC_k^{T} terms into dense workspace before sparse Horner evaluation. The 16-channel profile uses high-only coefficient projection and a separately compiled compensated final step. It trades extra projection workspace for throughput; it does not establish memory savings against every comparator.

For degree d, m stored nonzeros, n vertices, and q channels, the stated arithmetic work is $O(dmq + dnq^2)$. Narrow resident storage is $O(m + nq + dq^2)$, including the sparse operator, scales, coefficients, and reused states. Wide projection adds $O(dnq)$ workspace. These are constant-factor and memory-traffic optimizations; the asymptotic graph-work bound is unchanged.

6. Relation to prior work

Polynomial evaluation, sparse Horner recurrences, exact ordered fusion of linear stages, and diagonal similarity are established ideas. Classical matrix balancing, including LAPACK's DGEBAL routine, is important prior art [1]. K GraphFuse's fixed, sparse, dyadic row/column heuristic differs from a dense iterative balancing routine and carries no optimality claim.

Crouzeix and Palencia established a complete $(1+\sqrt{2})$-spectral-set bound for the numerical range [2]. A bivariate extension connects related estimates to Fréchet derivatives of matrix functions [4]. The current planner uses the constant-two complete inequality from OpenAI manuscript 325-01 as its stated premise [3]. OpenAI manuscript 325-02 motivates optimal similarity/positive-metric formulations; this implementation does not compute that optimal metric or transfer its condition bound to the heuristic diagonal scaling [5]. A separate OpenAI manuscript on the matrix multiplication exponent is not implemented and implies no finite-T4 speed result [6].

FMA semantics and the effects of floating-point operation order follow standard floating-point behavior documented by NVIDIA [7]. Compensation and mixed precision are execution choices, not exact-arithmetic certificates. The specific combination and performance behavior reported here are evaluated only under the experiment below.

7. Experimental design

7.1 Constructed family

The experiment uses one constructed asymmetric sparse family at $n = 10^4, 10^5, 10^6$, with $q = 4$ and a separate $n = 10^5, q = 16$ case. The operator has alternating diagonal entries -0.2 and -0.1 and alternating asymmetric directed couplings 1 and $1/64$. The selected D alternates 1 and $1/8$, giving $\kappa_2(D) = 8$. Each case uses a degree-48 matrix polynomial with geometrically decaying rational coefficients. Sensitivity cases use E diagonal with alternating ± 1 and parameter interval $[-1/64, 1/64]$. Their timings replay $\theta = 0$; the sensitivity accuracy checks additionally include both interval endpoints.

This is a deliberately chosen synthetic family. The six cases are related sizes and feature widths of the same construction, not six independent graph families.

7.2 Sources, controls, and profiles

The main timed implementation is frozen as *source/product_v1*; the separate wide follow-up is *source/wide_v1*. The baseline controls use a 10^{-8} compiler truncation target, the similarity-aware main plans target 10^{-16}, and the high-only wide profile targets 10^{-15}. The controls are FP64 CuPy 14.2.0 and PyTorch 2.11.0+cu128 on the same Tesla T4, independently tuned for packing and graph capture where supported. Fourteen configurations were recorded per main case and eight in the wide follow-up.

Seven pilot samples per configuration preceded five randomized held-out confirmation blocks of 31 synchronized samples per selected method. The fastest accuracy-eligible pilot profile was selected; the comparison denominator is the fastest accepted held-out control. The selection policy and public 10% latency threshold were reconstructed after exploratory results, and the wide follow-up followed earlier wide-profile losses. This creates selection bias not represented in the intervals.

The selected profiles were fused FMA for the narrow value-only cases, mixed value/derivative compensation for the 100,000-vertex sensitivity case, full compensation for the million-vertex sensitivity case, and BLAS projection with a compensated final step for the 16-channel value case. Selection is hardware- and workload-specific, not a universal dispatcher.

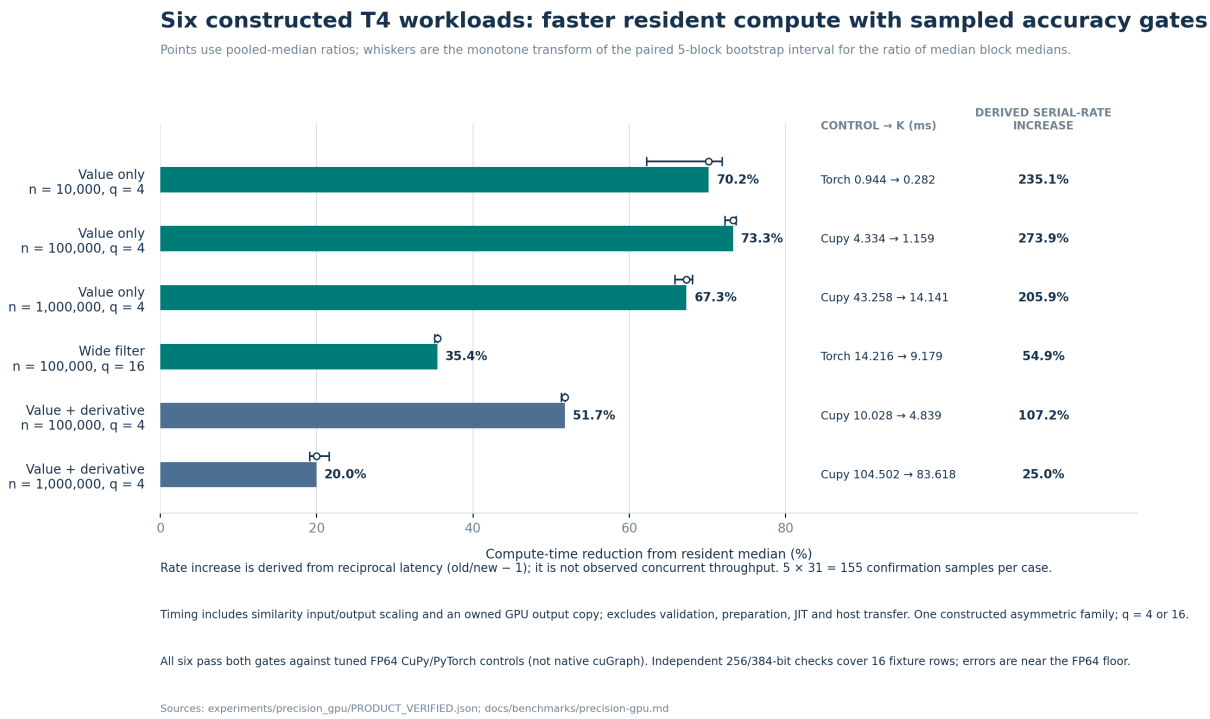
7.3 Accuracy and speed gates

For each sampled value/derivative output, the accuracy screen requires every recorded Frobenius and maximum-absolute error component to be no worse than the fastest accepted control, with at least one component better. The reported “largest Frobenius component reduction” is a summary of sampled numerical errors, not a per-vertex or model-quality improvement. The speed gate requires at least 10% lower pooled median latency, the lower endpoint of the reported block interval above one, and p95 no worse.

The pooled-median ratio is the ratio of medians from all 155 held-out samples. The reported interval instead bootstraps the ratio of median block medians. These estimators differ; the interval need not contain the pooled-median ratio. Five blocks from one run characterize within-run variation, not cross-run or cross-device uncertainty, and the interval is not corrected for profile selection.

Timing is resident validated-input compute, including the owned GPU output and input/output similarity scaling. It excludes validation, host transfer, compilation/JIT and preparation. No isolated peak-memory, energy, end-to-end application, or cloud-bill measurement was made.

Figure 2. *Latency reductions and reciprocal-latency rate changes.*



8. Results

All six selected T4 cases passed the recorded combined gates against their fastest accepted FP64 CuPy or PyTorch controls. The rate-change column is calculated as control median divided by K median, minus one. It is a reciprocal-latency transformation of the timing result, not a separate measured system-throughput result. See metric definitions for the formulas and interpretation.

Workload	Selected profile / control	Control median (ms)	K median (ms)	Latency reduction	Reciprocal-latency rate increase	Median ratio (block 95% interval)	Control / K p95 (ms)
10k × 4, filter	FMA / PyTorch	0.944	0.282	70.16%	235.12%	3.35× (2.65–3.55)	1.057 / 0.400
100k × 4, filter	FMA / CuPy	4.334	1.159	73.26%	273.95%	3.74× (3.60–3.79)	4.369 / 1.300
1M × 4, filter	FMA / CuPy	43.258	14.141	67.31%	205.91%	3.06× (2.93–3.13)	44.172 / 15.176
100k × 16, filter	Wide final / PyTorch	14.216	9.179	35.43%	54.88%	1.55× (1.54–1.55)	14.336 / 9.290
100k × 4, value + derivative	Mixed / CuPy	10.028	4.839	51.75%	107.24%	2.07× (2.05–2.07)	10.109 / 4.888
1M × 4, value + derivative	Compensated / CuPy	104.502	83.618	19.98%	24.98%	1.25× (1.24–1.28)	106.124 / 85.518

All pooled-median ratios meet the 10% reduction threshold; all interval lower endpoints exceed one and all selected p95 values are lower. The ratios and intervals are rounded from the audited PRODUCT_VERIFIED.json record.

Every full output was checked against degree-48 FP64 CPU evaluation; normalized differences were at most 10^{-13}. That CPU comparison covers all output entries but is itself rounded. Independent 256- and 384-bit sparse Horner references checked the first 16 rows for large fixtures using 64- and 128-row influence windows; those windows agreed below 10^{-65}. The finite influence-cone argument supports those

sampled rows only. It does not establish arbitrary-precision truth for every output row or for other operators.

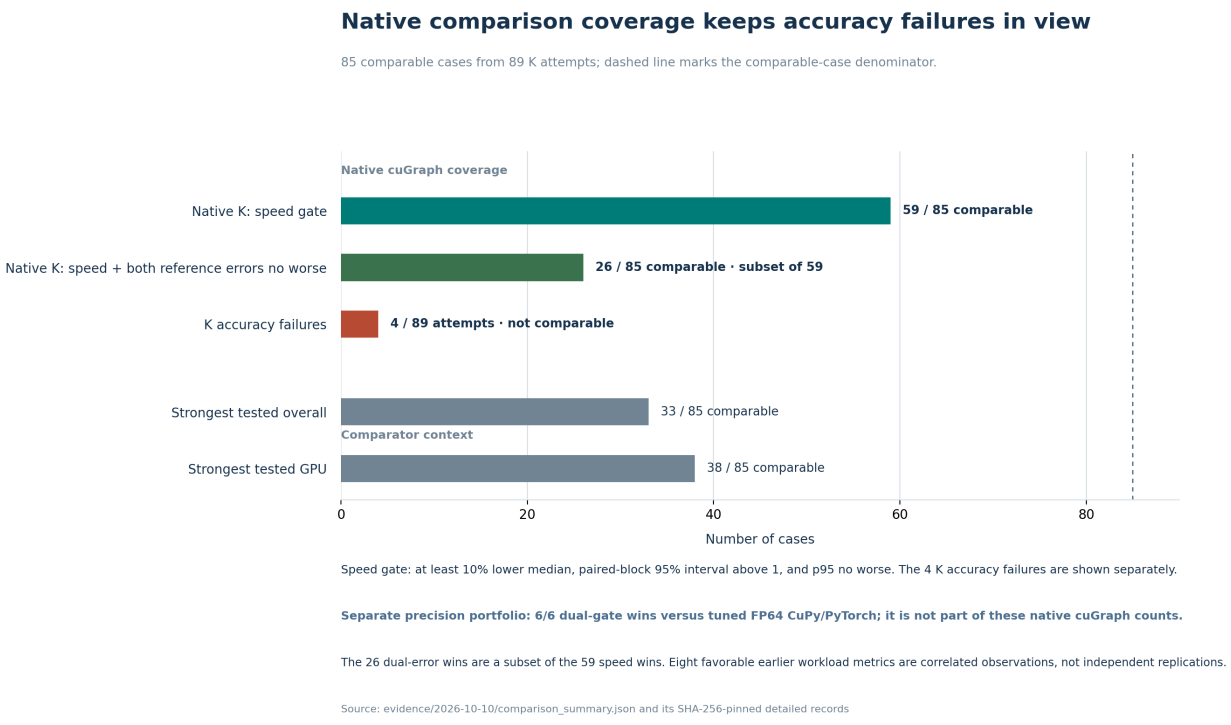
The sampled errors were near the FP64 floor. The largest sampled Frobenius component decreased by 0.0077%, 1.9316%, 10.0281%, 1.4723%, 28.7375%, and 19.9051%, respectively, in table order. Such small absolute errors mean a relative percentage can be numerically noticeable without implying a material downstream application gain. In the million-vertex sensitivity case, a faster mixed profile failed an error component; the selected fully compensated profile was slower but passed the screen.

8.1 Separate native PageRank context

The precision experiment has no native-cuGraph control. A separate K GraphFuse PageRank/ranking study used different source, operators, error contracts, and candidate pools; it must not be pooled with the six cases above. That study records 15 suites, 111 records, 2,190 configurations, and 104 predeclared cases. Of 89 attempted native comparisons, 85 were comparable and 4 K cases lacked accepted accuracy. A strong speed gate passed in 59/85 comparable cases (59/89 attempts when failures count as non-wins); 26/85 comparables (26/89 attempts) also had aggregate L_1 and maximum-relative errors no worse. The strongest-tested-control counts were 38/85 for GPU and 33/85 overall, including measured CPU alternatives. These denominators and gates belong only to that study.

The separate study also has losses: native cuGraph was 2.72× faster on Graph500-22; the later datagen confirmation measured K at 56.089 ms versus 54.667 ms for native; and full file-import and changing-graph rebuild workflows favored native cuGraph. The earlier 19-workload suite passed zero strongest GPU gates under its protocol. K GraphFuse does not implement BFS, SSSP, connected components, triangle counting, or Louvain. The native context provides scope and negative evidence, not corroboration of the precision-GPU results.

Figure 3. *Coverage of the separate native-comparison study.*



9. Limitations and threats to validity

Workload coverage. The experiment uses one synthetic alternating directed family. It does not measure public graph datasets, arbitrary sparse matrices, broad coefficient distributions, or other graph algorithms. At most, it motivates tests on similarly asymmetric linear filters and diagonal sensitivities.

Selection and uncertainty. Profile selection follows pilots and exploratory results; the wide follow-up was designed after earlier wide profiles lost. The public selection policy was reconstructed after exploration. The reported block interval covers within-run variability only and does not correct selection or multiple comparisons. There is one T4 setup and no independent cross-device replication.

Accuracy scope. The full-output reference is FP64 CPU arithmetic, not arbitrary precision. High-precision checks cover only the first 16 rows of the specified fixtures. GPU accuracy is not proven for every possible input or every parameter in the interval. The selected errors are near the representational floor, where the output format bounds achievable improvement.

Certificate scope. The transfer theorem is elementary and proved above. The complete constant-two Crouzeix result is a supplied theorem premise from a pinned external manuscript; it was not independently reconstructed here. The dyadic balancing rule is heuristic and not optimal. Exact truncation and coefficient bounds exclude execution rounding unless a separate floating-point proof is supplied.

Timing scope. Reported times exclude setup and JIT, host transfer, input validation, graph loading, output serialization, and full application workflows. Rate increases are reciprocals of per-call latencies. They do not measure resource utilization, concurrent capacity, energy, cloud cost, or end-user benefit. The results use frozen timed sources; the subsequently repaired checked API passed 29 core plus 14 upgrade checks but has no new full latency confirmation.

External validity and priority. The manuscript does not establish a new theorem about balancing, a globally optimal metric, global state of the art, original-world priority for the individual engineering ideas, peer review, or independent replication. The broader PageRank results are from distinct experiments and have documented losses.

10. Applications and adoption criteria

The best-fit hypotheses are repeated low-channel sparse linear graph filters, asymmetric diffusion-style polynomials, and repeated sensitivity evaluations with a diagonal node parameter. A candidate application should first match the graph operator, coefficient order, channel width, precision, parameter range, and error norm. It should then compare the full request path—including setup, transfers, validation, and the current implementation—on representative data, with independent output checks and held-out cases.

The method may lose on already balanced operators, wider channels, different hardware, or workloads where preparation dominates. The current prototype has no universal metric search or automatic fallback router. A degree or speed win does not establish improved recommendation quality, recall, fraud detection, ranking quality, or any other downstream metric.

11. Conclusion

K GraphFuse’s experimental prototype combines a provable coordinate-error transfer, a simple dyadic balancing heuristic, fused sparse Horner evaluation, and selective GPU compensation. On one constructed asymmetric family, six selected Tesla T4 workloads passed the recorded sampled-accuracy and speed gates against tuned FP64 CuPy/PyTorch controls, with 19.98%–73.26% lower pooled-median latency and corresponding 24.98%–273.95% higher derived reciprocal-latency rate. The latter is computed from latency ratios, not measured throughput. This is evidence for a narrow numerical workload, not a native-cuGraph win or a general graph-processing claim. Broader adoption requires independent hardware and workload replication, complete-path timing, and accuracy validation under the target application’s own contract.

Reproducibility and artifact status

The audited record is *experiments/precision_gpu/PRODUCT_VERIFIED.json*; the mathematical derivation and premise ledger are *experiments/precision_gpu/DERIVATION.md* and *experiments/precision_gpu/RESEARCH.md*. Timed sources are *experiments/precision_gpu/source/product_v1* and *source/wide_v1*. The later checked API at *experiments/precision_gpu/source/repaired_api* passed 29 core plus 14 upgrade checks but does not retime the six cases. The repository's offline evidence audit can check recorded results without launching a GPU benchmark. No private host paths, account screenshots, or contact details are included. No DOI, publication venue, or acceptance is asserted for this manuscript.

References

1. LAPACK. "DGEBAL: Balance a general matrix." LAPACK reference documentation. Netlib documentation. Accessed 10 October 2026.
2. M. Crouzeix and C. Palencia. "The Numerical Range is a $(1+\sqrt{2})$-Spectral Set." *SIAM Journal on Matrix Analysis and Applications* 38(2), 649–655 (2017). doi:10.1137/17M1116672.
3. OpenAI. "A Direct Proof of the Complete Crouzeix Inequality." Mathematics preprint 325-01, repository snapshot *fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb*, 26 September 2026. Pinned manuscript. Supplied premise; not independently re-proved in this work.
4. M. Crouzeix and D. Kressner. "A bivariate extension of the Crouzeix–Palencia result with an application to Fréchet derivatives of matrix functions." arXiv:2007.09784 (2020). arXiv.
5. OpenAI. "The Complete Crouzeix Theorem." Mathematics preprint 325-02, repository snapshot *fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb*, 23 September 2026. Pinned manuscript. Motivational context; its optimal metric is not implemented.
6. OpenAI. "An Upper Bound of 9/4 for Matrix Multiplication Exponent." Mathematics preprint 107-01, repository snapshot *fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb*, 2 October 2026. Pinned manuscript. Not implemented.
7. NVIDIA. "Floating Point and IEEE 754 Compliance for NVIDIA GPUs." CUDA documentation. Accessed 10 October 2026.
8. U. Sinha. *K GraphFuse: Sparse Graph Polynomial Plans and CUDA Execution*, software v0.1.0, released 10 October 2026. Repository. The experimental sources described here are separately versioned under *experiments/precision_gpu/*.